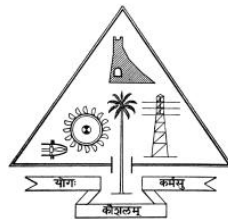


DISSERTATION PHASE ONE
on
DIABETIC PREDICTION USING 2DNN WITH
DATA PRIVACY PROTECT

Submitted by
ANJANA P T (TCR24CSCE02)
Under the Guidance of
MS ANUSREE RADHAKRISHAN

to
APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the award of the Degree of
Master of Technology
in
Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GOVERNMENT ENGINEERING COLLEGE, THRISSUR
KERALA, INDIA

November 2025

DECLARATION

I, **ANJANA P T (TCR24CSCE02)**, hereby declare that this M.Tech project report titled “**DIABETIC PREDICTION USING 2DNN WITH DATA PRIVACY PROTECT**”, submitted to **APJ Abdul Kalam Technological University**, Kerala, is a bonafide record of work carried out by me under the supervision of **MS ANUSREE RADHAKRISHAN** , Department of Computer Science and Engineering, Government Engineering College, Thrissur.

This submission represents my ideas and the work carried out by me. Where ideas or works of others have been included, I have duly cited and acknowledged them. This report has not been submitted to any other University or Institute for any degree or diploma.

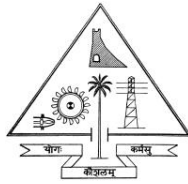
Place: Thrissur

ANJANA P T

Date: November 2025

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
GOVERNMENT ENGINEERING COLLEGE, THRISSUR**

CERTIFICATE



This is to certify that the M.Tech project report titled **“DIABETIC PREDICTION USING 2DNN WITH DATA PRIVACY PROTECT”**, submitted by **ANJANA P T (TCR24CSCE02)** to **APJ Abdul Kalam Technological University**, Kerala, is a bonafide record of the project work carried out by him under my supervision in the Department of Computer Science and Engineering, Government Engineering College, Thrissur for the academic year **2024–2025**.

This report has not been submitted to any other University or Institute for any purpose.

MS ANUSREE RADHAKRISHAN

Project Guide

Dr. Swaraj K P

Head of the Department

Examiner

Prof. Dr. Giles M. P

PG Coordinator

GEC, Thrissur

ACKNOWLEDGEMENT

First and foremost, I express my heartfelt gratitude to God Almighty for His abundant blessings, guidance, and grace throughout the successful completion of this M.Tech project.

I convey my sincere thanks to **Dr. Solaman P A**, Principal, Government Engineering College, Thrissur, for providing me with the opportunity and the necessary facilities to carry out this project work.

I wish to express my deep sense of gratitude to **Prof. Dr. Swaraj K. P**, Head of the Department of Computer Science and Engineering, for his support, encouragement, and valuable administrative guidance throughout the duration of this project.

I am profoundly grateful to my guide, , for his invaluable guidance, timely suggestions, continuous encouragement, and constant support. His expertise and motivation played a pivotal role in shaping this work into its present form.

I also extend my sincere thanks to **Prof. Dr. Giles M. P**, PG Coordinator, Department of Computer Science and Engineering, for his support, insightful suggestions, and continuous encouragement during the course of my M.Tech program.

My heartfelt thanks are due to all the teaching and non-teaching staff members of the Department of Computer Science and Engineering, Government Engineering College, Thrissur, for their cooperation, assistance, and academic support.

I am also grateful to my classmates and friends for their support, constructive feedback, and encouragement, which greatly helped me throughout the project.

Finally, I express my deepest gratitude to my parents and family members for their unconditional love, constant motivation, and unwavering support, which have been the foundation of all my achievements.

ANJANA P T (TCR24CSCE02)

ABSTRACT

Server-Side Request Forgery (SSRF) is a severe security vulnerability in which attackers manipulate backend services to send unintended requests to internal or external endpoints. Microservice architectures, due to their dense internal communication, are particularly vulnerable to such attacks. Existing SSRF detection techniques primarily rely on signatures, heuristics, or labeled datasets, making them ineffective against zero-day or obfuscated SSRF attacks.

This project focuses on the initial stages required for an unsupervised graph-based SSRF detection framework. Specifically, the work completed includes microservice deployment, log collection, log parsing, dataset construction, and extensive feature extraction. Although the full anomaly detection model combining Graph Neural Networks and LSTM is not implemented, the dataset generated in this work forms the foundation for future spatio-temporal SSRF detection pipelines.

The output of this project includes a structured request-level dataset, node-level behavioral features, and edge-level semantic and statistical features. These datasets align with the preprocessing requirements found in recent GNN-based anomaly detection and deep-learning-based SSRF research.

Contents

DECLARATION	i
CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
1 INTRODUCTION	1
2 LITERATURE SURVEY	5
2.1 SSRF Vulnerabilities in Cloud and Microservices	5
2.2 Microservice Security and Anomalies	6
2.3 Graph Neural Networks for Anomaly Detection	7
2.4 Deep Learning for Log Analysis	8
2.5 Spatio-Temporal Learning in Distributed Systems	8
2.6 Need for Unsupervised Detection	9
2.7 Dataset Construction Challenges	9
2.8 Summary of Literature	9
3 PROPOSED METHODOLOGY	11
3.1 Conceptual System Architecture	11
3.2 Log Collection Methodology	12
3.3 Log Parsing and Preprocessing	13
3.4 Node Feature Engineering	13
3.5 Edge Feature Engineering	14
3.6 Graph Construction Methodology	15
3.7 Conceptual Spatial Learning with GraphSAGE	15
3.8 Temporal Modelling with LSTM	15
3.9 Anomaly Detection via Unsupervised Learning	16

3.10	Summary of Proposed Methodology	16
4	IMPLEMENTATION	18
4.1	Microservice Environment Deployment	18
4.2	Traffic Generation Using Loadgenerator	19
4.3	Log Extraction and Consolidation	20
4.4	Python-Based Log Parsing Pipeline	20
4.5	Dataset Structuring and Cleaning	21
4.6	Node-Level Feature Engineering	21
4.7	Edge-Level Feature Engineering	22
4.8	Saving Feature Files and Dataset Outputs	22
4.9	Conceptual Design of SSRF Detection Architecture	23
4.10	Summary	23
5	RESULTS AND ANALYSIS	24
5.1	Request-Level Dataset Output	24
5.2	Kubernetes Log Output	25
5.3	Node-Level Feature Output	25
5.4	Edge-Level Feature Output	26
5.5	Summary of Extracted Results	26
6	CONCLUSION AND FUTURE WORK	28
	REFERENCES	31

List of Figures

3.1	Conceptual System Architecture for Unsupervised SSRF Detection	17
5.1	Extracted Request-Level Dataset After Parsing the Kubernetes Log	24
5.2	Raw Kubernetes Log Capturing Microservice-to-Microservice Communication	25
5.3	Node-Level Behavioural Features Extracted from the Structured Dataset	26
5.4	Edge-Level Communication Features for Microservice Interactions	26

CHAPTER 1

INTRODUCTION

The evolution of software architecture over the past decade has been marked by a significant transition from monolithic systems to microservice-based deployments. Microservices have introduced a paradigm shift in how modern applications are designed, developed, deployed, monitored, and scaled. Instead of operating as a single unified structure, microservices adopt a decentralized approach where functionalities are broken down into small, autonomous services. Each service runs independently, communicates through lightweight mechanisms such as HTTP or gRPC, and can be scaled horizontally based on demand. This architectural model has enabled organizations to develop systems that are more fault-tolerant, maintainable, and adaptable to rapidly changing requirements.

However, the benefits of microservices come with corresponding security challenges. The decomposition of a monolithic system into multiple networked components increases the number of internal communication pathways, API endpoints, and service-to-service interactions. As a result, the internal attack surface expands significantly. Threat actors exploit weaknesses in internal communication to launch sophisticated attacks that may not be visible to traditional perimeter-based security mechanisms. One such attack vector that has emerged as particularly dangerous is Server-Side Request Forgery (SSRF), a vulnerability that manipulates backend services into issuing unintended internal requests.

SSRF attacks leverage trusted server roles to access internal endpoints, sensitive cloud metadata URLs, administrative interfaces, internal APIs, and private resources that are otherwise inaccessible from outside the system. Because the requests originate from legitimate microservices, they bypass external firewalls, filtering mechanisms, and IP-based access control systems. This characteristic makes SSRF a uniquely powerful vulnerability in cloud-native and microservice environments. The severity of SSRF attacks has been emphasized in several studies, including the work of Al-Talak and Abbass [1], who show that even simple SSRF

payloads can cause high-impact internal data exposure.

The complexity of SSRF detection increases significantly in microservice ecosystems. Each service produces logs that describe requests it sends and receives, including URL paths, query parameters, HTTP methods, response codes, and latency measurements. While human operators can detect some abnormalities manually, the volume of log data generated within a microservice architecture quickly overwhelms manual monitoring approaches. Furthermore, SSRF attacks frequently use obfuscated URLs, indirect redirection chains, or payload-based evasion techniques. This makes signature-based or rule-based detection insufficient, as these techniques rely on predefined patterns that fail against zero-day or evolving SSRF variants.

In addition to obfuscation and rapid evolution, SSRF attacks are often indistinguishable from legitimate requests based on shallow inspection. Only deeper behavioural analysis, such as reviewing internal communication patterns over time, can reveal that a service has deviated from its typical communication behaviour. For example, a payment service suddenly interacting with an internal metadata endpoint is an anomalous pattern that would not be captured by signature-based systems. This highlights the need for an intelligent detection mechanism that can learn the natural behaviour of microservices and identify deviations automatically.

Graph Neural Networks (GNNs) have emerged as a powerful technique for learning structural patterns in data. Microservice architectures can be naturally represented as graphs where nodes correspond to services and edges represent service-to-service requests. GNNs such as GraphSAGE have been shown to effectively capture complex structural relationships in microservices, as demonstrated in studies like Fang et al. [2]. The ability to aggregate information from neighbouring services allows GNNs to learn contextual representations of microservice behaviour that are more expressive than traditional vector-based models.

Beyond structure, microservice behaviour evolves over time. Time-dependent changes such as load variations, service restarts, auto-scaling events, and temporary network congestion all influence communication patterns. Therefore, detecting anomalies requires a temporal understanding of how these patterns change. Jin et al. [4] highlight the importance of combining graph-based spatial learning with temporal learning techniques, particularly for anomaly detection tasks. Their survey of spatio-temporal GNN models confirms that many real-world attacks manifest as deviations in long-term behaviour rather than isolated events.

Although this project does not implement the full Graph Neural Network or LSTM-based temporal model, the motivation for future work is grounded in these research findings. Instead,

this project focuses on the essential foundational step: constructing a rich dataset capable of supporting such advanced detection systems. The dataset includes raw logs, parsed request-level records, node-level features, edge-level features, temporal context, and URL semantic analysis. These components represent the minimum requirements for future graph-based SSRF detection.

The dataset was constructed using logs collected from the Google Cloud Online Boutique microservice application. This environment provides a realistic distributed architecture composed of multiple independent services such as frontend, product catalog, recommendation, checkout, and payment. Each service generates HTTP logs that reveal how internal communication flows through the system. The logs were extracted using Kubernetes tools and subsequently parsed using Python scripts. Important features such as source service, destination service, HTTP method, URL path, response code, latency, payload size, and timestamp were extracted for each request.

The transformation of these logs into a structured dataset was a multi-stage process involving several steps. First, unstructured log text had to be parsed using regular expressions to extract meaningful fields. Second, requests were grouped by service to derive node-level behavioural features. Third, service-to-service interactions were analysed to compute edge-level features such as call frequencies, latency distributions, and URL semantic indicators. Finally, the dataset was organized into a structure amenable to graph construction.

This dataset provides a foundation for anomaly detection pipelines. Graph-based learning approaches such as those described in Ghadermazi et al. [3] require well-structured node and edge features to learn meaningful embeddings. Similarly, temporal models require logs to be grouped into time windows or sequential form, which is facilitated by the inclusion of timestamp information in the dataset. By extracting both structural and semantic features, this project ensures that the dataset captures the key elements needed to detect SSRF attacks in the future.

In summary, this introductory chapter highlights the importance of SSRF detection in microservice architectures, the limitations of existing detection methods, and the motivation for adopting graph-based and temporal approaches. While the present work focuses solely on dataset construction and feature engineering, it is motivated by strong evidence in the literature regarding the effectiveness of GNNs and temporal models in anomaly detection tasks. The remaining chapters detail the theoretical foundations, related research, methodology, imple-

mentation details, and dataset construction process.

CHAPTER 2

LITERATURE SURVEY

The evolution of distributed computing and modern cloud-native architectures has led to the widespread adoption of microservice-based application design. Microservices divide system functionality into independently deployable services that communicate through well-defined APIs, often using lightweight protocols such as HTTP and gRPC. This shift has improved scalability, resilience, and maintainability in large-scale applications. However, it has also introduced new layers of complexity in the security domain. The distributed nature of microservices means that internal components communicate frequently across potentially hundreds of service boundaries, increasing the attack surface exponentially.

A comprehensive understanding of the vulnerabilities that arise in microservice architectures is essential for detecting anomalies such as Server-Side Request Forgery (SSRF) attacks. SSRF manipulation tricks backend services into sending unauthorized internal requests, potentially exposing sensitive internal endpoints or cloud metadata services. Traditional signature-based systems often fail to detect these attacks due to their dynamic nature, payload obfuscation, and dependence on evolving attack vectors. This literature survey presents an in-depth exploration of existing work related to SSRF detection, graph-based anomaly detection, microservice-specific security analysis, deep learning techniques for log analysis, and spatio-temporal modeling in distributed systems. The survey builds the theoretical foundation for the dataset construction and methodological design presented in later chapters.

2.1 SSRF Vulnerabilities in Cloud and Microservices

Server-Side Request Forgery is recognized as one of the most dangerous vulnerabilities in modern distributed systems. Its categorization as a critical threat has been emphasized in various industry reports and academic studies. SSRF attacks leverage the trusted execution environ-

ment of backend services to interact with unauthorized internal endpoints. In traditional monolithic systems, SSRF attacks are harmful but limited to the compromised system. In contrast, microservice-based environments amplify the impact by providing numerous internal endpoints and intra-cluster communication paths.

Al-Talak and Abbass [1] conducted one of the earliest deep-learning-based studies focused on SSRF detection. Their research highlighted that SSRF attacks often manifest through anomalous URL structures, suspicious query patterns, and access to internal IP addresses. Their study introduced an LSTM-based model trained on URL sequences, demonstrating that sequential modeling could capture temporal patterns related to SSRF attempts. However, their approach required labeled datasets, which are difficult to obtain in real-world settings where SSRF attacks are rare and often undisclosed. Despite this limitation, their work is significant because it introduced the idea that SSRF attacks follow detectable behavioural patterns and that these patterns can be learned from historical data.

The increasing number of SSRF attacks in cloud-native environments, particularly those targeting AWS, Google Cloud, and Azure metadata endpoints, has raised further concern. Industry reports have shown that metadata URLs such as ‘169.254.169.254’ are frequent targets for SSRF-based privilege escalation. These endpoints expose highly sensitive information like IAM credentials, network configurations, and access tokens. Research into cloud SSRF cases suggests that attackers frequently exploit blind SSRF, where the service does not return output directly to the attacker, making the detection significantly harder. These studies collectively indicate that SSRF detection mechanisms must focus on deep behavioural and contextual analysis rather than superficial signature matching.

2.2 Microservice Security and Anomalies

The transition to microservice architectures introduces several new classes of anomalies that are often difficult to detect using traditional intrusion detection techniques. Microservices communicate through numerous internal HTTP requests, and these communication pathways form a complex internal graph composed of directed edges representing requests. This dynamic internal communication graph changes with user activity, load balancing, scaling events, deployments, and failures. Anomalies in microservices can include performance faults, unauthorized access, lateral movement, dependency misconfigurations, and communication irregularities.

Several studies have outlined the unique challenges posed by microservices. The distributed

nature makes it difficult to establish a stable baseline of normal behaviour. Moreover, services often scale dynamically, which makes their communication patterns fluctuating. Fang et al. [2] conducted extensive research showing that microservice calls can be represented as graph structures that evolve over time. Their work demonstrated that traditional monitoring tools, including Prometheus and Jaeger, provide raw metrics but lack the contextual learning ability required for anomaly detection. This limitation led to the development of Graph Neural Networks for microservice anomaly detection.

An important observation from Fang et al. is that microservices exhibit relational patterns. For instance, the frontend service always interacts with the product catalog service, and the checkout service interacts with payment and shipping services. These stable relational dependencies form a graph structure that remains relatively consistent. When a service starts communicating with an unusual target, performing unexpected request volumes, or generating unusual URL patterns, this relational graph deviates from its typical form. Modeling this relational structure is essential for detecting anomalies such as SSRF, where a service begins accessing unexpected internal endpoints.

2.3 Graph Neural Networks for Anomaly Detection

Graphs naturally represent relationships in distributed systems. Graph Neural Networks have gained attention due to their ability to model structural patterns in complex interconnected systems. Among various GNN architectures, GraphSAGE is widely recognized for its inductive learning capabilities, meaning it can make predictions on unseen nodes or graph structures. This is crucial for microservice environments where services may scale or change dynamically.

Graph-based anomaly detection has been explored extensively in recent years. Ghadermazi et al. [3] introduced the Graph Transformer Autoencoder (GTAE) model for intrusion detection. Their approach combined Transformer-based attention with graph autoencoders, enabling the model to learn high-level structural embeddings. Their work highlights the importance of reconstructive learning for anomaly detection, where anomalies produce lower reconstruction quality due to the model's inability to represent unusual patterns.

Other graph-based detection approaches also emphasize reconstructive error as a strong anomaly indicator. However, many graph models lack temporal awareness. Since SSRF attempts often unfold over time, combining graph-based spatial learning with temporal models such as LSTMs or GRUs becomes essential for capturing attacks that slowly deviate from nor-

mal communication patterns.

2.4 Deep Learning for Log Analysis

Deep learning techniques have been increasingly applied to log-based anomaly detection. Approaches range from simple LSTM-based sequence models to complex attention-based transformers. Log entries in microservices contain rich contextual information. Each request includes multiple fields such as timestamp, service name, method, URL, payload size, and response code. Sequential deep learning models have been used to detect patterns in these logs.

For SSRF-specific log analysis, researchers have explored URL-based anomaly detection. URL tokenization, embedding, and sequence modeling using LSTMs have been effective in detecting abnormal URL sequences. However, sequence-only models ignore the structural relationships among microservices, which are crucial for detecting anomalies that involve inter-service behaviour changes.

In contrast, graph-based log analysis addresses this limitation by combining structure with content. GNN models aggregate structural neighbourhood information with log-derived features, allowing them to capture both spatial and semantic characteristics. The downside of many deep learning log-based approaches is their dependency on large labeled datasets, which are rarely available in operational settings.

2.5 Spatio-Temporal Learning in Distributed Systems

Distributed systems involve behaviours that evolve over time. Anomaly detection in such settings requires models that can capture both spatial relationships and temporal patterns. Jin et al. [4] conducted an extensive survey of Graph Neural Networks for time series analysis. Their findings emphasize the importance of combining GNNs with temporal models when working with dynamic graphs. The survey categorizes spatio-temporal GNN architectures into recurrent, convolutional, and attention-based models.

Recurrent spatio-temporal models combine GNNs with recurrent neural networks such as LSTMs to capture sequential dynamics. These models are particularly suitable for microservices where communication patterns change with user activity. Anomalies like SSRF may arise randomly or escalate gradually, requiring temporal context for accurate detection.

The survey also highlights that many anomalies in distributed systems manifest in subtle

deviations from typical patterns. For example, a sudden spike in internal communication to an unusual endpoint can be preceded by minor deviations that only temporal models can detect. This reinforces the importance of constructing datasets that preserve temporal context, making time-indexed logs essential for future anomaly detection.

2.6 Need for Unsupervised Detection

One of the major challenges in SSRF detection is the scarcity of labeled datasets. Real-world SSRF incidents are often not shared publicly due to their sensitive nature. Furthermore, SSRF attacks vary greatly in pattern and payload structure, making supervised learning approaches unreliable.

Unsupervised models address these challenges by learning normal behaviour without requiring malicious examples. Autoencoders, clustering-based methods, and graph reconstruction approaches are widely used unsupervised techniques. Studies like GTAE [3] illustrate how reconstructive learning can flag anomalies based on deviation from learned patterns. However, unsupervised techniques require high-quality feature extraction to effectively differentiate between normal and abnormal communication.

2.7 Dataset Construction Challenges

A common limitation in existing microservice anomaly detection research is the absence of publicly available datasets. Most studies rely on proprietary datasets or simulated environments. As a result, many published works emphasize the importance of designing realistic microservice environments for capturing meaningful logs.

Building a dataset suitable for graph-based anomaly detection requires capturing both request-level details and structural relationships. Studies repeatedly highlight the importance of extracting service-level summaries (node features) and interaction-level summaries (edge features). The feature extraction performed in this project aligns closely with these requirements.

2.8 Summary of Literature

The literature surveyed highlights several critical themes. SSRF attacks are difficult to detect using signature-based systems due to their evolving nature and behavioural complexity. Mi-

crosservice architectures introduce structural relationships that can be modeled effectively using Graph Neural Networks. Temporal modelling is essential for capturing dynamic patterns. Deep learning techniques offer effective approaches but rely heavily on well-prepared datasets. The absence of publicly accessible SSRF datasets underscores the need for research efforts focused on dataset creation and feature engineering.

This project contributes to addressing this gap by producing a comprehensive dataset suitable for future GNN-LSTM based SSRF detection models.

CHAPTER 3

PROPOSED METHODOLOGY

The goal of this project is to develop the foundational dataset for an unsupervised Server-Side Request Forgery (SSRF) detection framework tailored to microservice environments. While the full machine learning pipeline, including Graph Neural Networks and LSTM-based temporal learning, is reserved for future work, the methodological design presented in this chapter describes the complete conceptual system architecture and the rationale underlying its components. The dataset construction and feature extraction work implemented in this project are guided by this comprehensive methodology. The chapter therefore presents an in-depth explanation of how microservice logs can be transformed into graph-structured datasets that capture both spatial and temporal behaviour patterns essential for detecting SSRF anomalies.

The proposed methodology is anchored in observations made in existing research. Microservices form directed communication networks that resemble evolving graphs, and their behaviour unfolds over time, as reflected in the findings of Fang et al. [2] and Jin et al. [4]. Furthermore, reconstructive learning frameworks such as GTAE [3] demonstrate the need for structured node and edge representations. Motivated by these insights, the conceptual system architecture combines log collection, parsing, feature engineering, graph construction, spatial representation learning, temporal sequence modeling, and anomaly scoring as an end-to-end SSRF detection framework. Only the first three stages—log handling and feature extraction—are implemented in this work, but the full pipeline is preserved as a theoretical foundation.

3.1 Conceptual System Architecture

The conceptual system architecture models the behaviour of microservices as a spatio-temporal graph learning problem. This framework views each microservice as a node and every HTTP

request between services as a directional edge. The structural topology of the microservice environment, combined with timestamped logs, forms a dynamic sequence of graphs that evolve across time. These graph snapshots, once enriched by extracted features, provide a rich representation of microservice behaviour.

The architecture is composed of several layers. The first layer captures raw logs from the microservice environment. These logs contain critical information about service communication, including timestamps, URLs, response codes, and latency. The second layer transforms raw logs into structured datasets by parsing and extracting meaningful fields. The third layer constructs node-level and edge-level features, which constitute the essential components of graph-based modelling. Subsequent layers, although not implemented in this project, conceptually apply Graph Neural Networks to learn structural embeddings and LSTM networks to learn temporal patterns. Finally, an anomaly detection layer uses reconstructive learning or distance-based scoring to identify deviations that may indicate SSRF activity.

The entire conceptual architecture is illustrated in Figure 3.1. This figure presents a high-level overview of the proposed SSRF detection system and shows how the implemented dataset forms the foundation for GNN-based anomaly detection.

3.2 Log Collection Methodology

Microservice environments generate extensive logs that reflect operational behaviour. To design an anomaly detection system, it is essential to capture data that represent authentic communication patterns. The log collection methodology begins with deploying the Google Cloud Online Boutique microservice application. This application consists of multiple services such as frontend, product catalog, checkout, shipping, payment, cart, and recommendation services, all communicating through HTTP requests.

Kubernetes serves as the container orchestration platform, and it provides access to logs through the `kubectl logs` command. Each microservice, deployed as a pod, produces its own set of logs. These logs include timestamps, sender service names, recipient service names, URL paths, HTTP methods, latency values, status codes, and payload sizes. The combination of these fields provides insights into both structural patterns and request semantics.

To simulate realistic user interactions, a load generator is deployed alongside the microservices. The load generator continuously invokes various endpoints, triggering internal communication across services. This ensures that logs include a wide variety of request flows,

capturing both common and rare interactions. The goal is to collect logs that enable the construction of dynamic graphs reflecting normal system behaviour, as highlighted by Fang et al. [2].

The log collection process ensures that temporal context is preserved through timestamps. This temporal dimension will be essential in future work, where logs will be grouped into time windows to form graph sequences. The structure of the collected logs forms the backbone for subsequent parsing and feature extraction.

3.3 Log Parsing and Preprocessing

The logs collected from Kubernetes are unstructured textual data. Parsing these logs involves extracting meaningful attributes needed for graph-based modeling. A Python script using regular expressions was developed to identify and parse each log entry. This stage is similar to preprocessing steps described in Al-Talak and Abbass [1], where logs are transformed into structured representations suitable for deep learning analysis.

Each log record is transformed into a structured row with attributes such as timestamp, source service, destination service, URL, HTTP method, response code, latency in milliseconds, and payload size. These structured records form a request-level dataset, which represents a detailed and chronological record of internal microservice communication.

The parsing process focuses on accuracy and completeness. Timestamp extraction ensures that temporal ordering is preserved, allowing future work to group logs into time windows. Service names are extracted to serve as node identifiers. The URL field is captured in full, as URL semantics are proven to contain crucial SSRF indicators, as shown by Al-Talak and Abbass [1]. Latency and payload size provide quantitative measures of communication performance, which can be used to detect unusual patterns. Status codes offer insights into error behaviours, which may correlate with SSRF attempts where unauthorized endpoints cause error responses.

The parsing process transforms unstructured text into a structured dataset stored as a CSV file, forming the basis for further feature engineering and graph construction.

3.4 Node Feature Engineering

Node features describe the behaviour of individual microservices. By aggregating all the requests the service sends or receives, a behavioural summary can be derived. Fang et al. [2]

emphasize that node features are essential for capturing the operational tendencies of microservices.

Node features extracted include the total number of requests associated with each service, the distribution of incoming versus outgoing calls, statistical properties such as average latency, maximum latency, minimum latency, and response code distributions. The diversity of URLs accessed by a service reflects whether it interacts with a wide range of endpoints or follows stable patterns. Payload size statistics and slow-call ratios contribute to understanding the service load and possible resource bottlenecks.

These node features provide a comprehensive summary of how each service behaves over the dataset’s time span. If a service begins interacting with unusual endpoints or showing latency spikes, such deviations will be reflected in its node-level feature vector. This makes node features particularly useful for future Graph Neural Network learning.

3.5 Edge Feature Engineering

While node features describe individual services, edge features capture the interactions between pairs of services. Edge features are crucial for identifying deviations in service-to-service communication, which is particularly relevant for SSRF attacks. SSRF attempts often manifest as unauthorized or unexpected communication between services. Therefore, capturing the properties of these connections is essential.

Edge features extracted include call frequency between two services, latency distributions, payload size statistics, counts of unique URLs used across the connection, error rate for the connection, and URL semantic indicators such as URL length, slash count, presence of internal IP addresses, or suspicious keywords. These features help in constructing an enriched representation of every service-to-service relationship.

As described in Fang et al. [2], relationships between services remain relatively stable under normal operation. When deviations occur, such as a service contacting an internal metadata endpoint or an external IP address, the edge feature vector will reflect such anomalies. This makes edge features essential for graph-based anomaly detection.

3.6 Graph Construction Methodology

Once node and edge features are extracted, the microservice communication dataset can be represented as a graph. Each service is treated as a node, and each communication between services is represented as a directional edge. The graph structure models the internal architecture of the microservice system.

To incorporate temporal dynamics, graph construction can be performed at regular intervals. For example, logs can be grouped into five-minute windows, and a separate graph snapshot can be constructed for each window. This approach captures the evolution of microservice communication patterns, enabling temporal modelling in future work. Jin et al. [4] emphasize the value of such dynamic graph modeling for anomaly detection.

By constructing graphs that reflect service interactions, the dataset becomes suitable for applications involving Graph Neural Networks and other graph-based learning techniques. This step bridges the gap between raw log data and the structured graph representations required for advanced modelling.

3.7 Conceptual Spatial Learning with GraphSAGE

GraphSAGE is chosen as the conceptual GNN architecture for the proposed system due to its inductive learning capabilities. Unlike traditional GNNs, GraphSAGE can generalize to unseen nodes or subgraphs, making it suitable for dynamic microservice environments where services may scale or change.

GraphSAGE learns embeddings by aggregating node and edge features from neighbouring nodes. This allows the model to capture contextual relationships, which is crucial in detecting SSRF anomalies where unusual communication emerges between services. Fang et al. [2] demonstrated the effectiveness of using GraphSAGE for microservice anomaly detection.

Although GraphSAGE is not implemented in this project, designing the dataset with GraphSAGE in mind ensures compatibility for future extensions.

3.8 Temporal Modelling with LSTM

Temporal modelling is essential for detecting SSRF anomalies that evolve over time. LSTM networks are capable of learning long-term dependencies and identifying gradual deviations

from normal behaviour. Jin et al. [4] emphasize that many anomalies manifest through changes that are not instantaneous but progressive.

By applying LSTM to sequences of graph embeddings, the model can learn behavioural trajectories. Such trajectories help in recognizing abnormal shifts. This temporal analysis layer remains conceptual in this project but provides direction for future work.

3.9 Anomaly Detection via Unsupervised Learning

Unsupervised anomaly detection techniques, such as autoencoder reconstruction error, provide a powerful mechanism for identifying deviations without labeled data. SSRF attacks are rare and often undocumented, making unsupervised learning more practical and effective.

In the proposed system, after spatial and temporal embeddings are generated, an autoencoder attempts to reconstruct normal behaviour. Higher reconstruction errors indicate anomalies. Ghadermazi et al. [3] demonstrate that reconstruction-based models are effective for intrusion detection.

Although anomaly detection is not implemented in this project, the dataset and methodology are designed to support such techniques.

3.10 Summary of Proposed Methodology

The proposed methodology integrates log collection, parsing, feature extraction, graph construction, spatial learning, temporal modeling, and anomaly scoring into a unified system for SSRF detection. This project implements the first stages: log handling, parsing, structured dataset creation, and feature extraction. These components form the foundation for future spatio-temporal SSRF detection using GNNs and LSTMs.

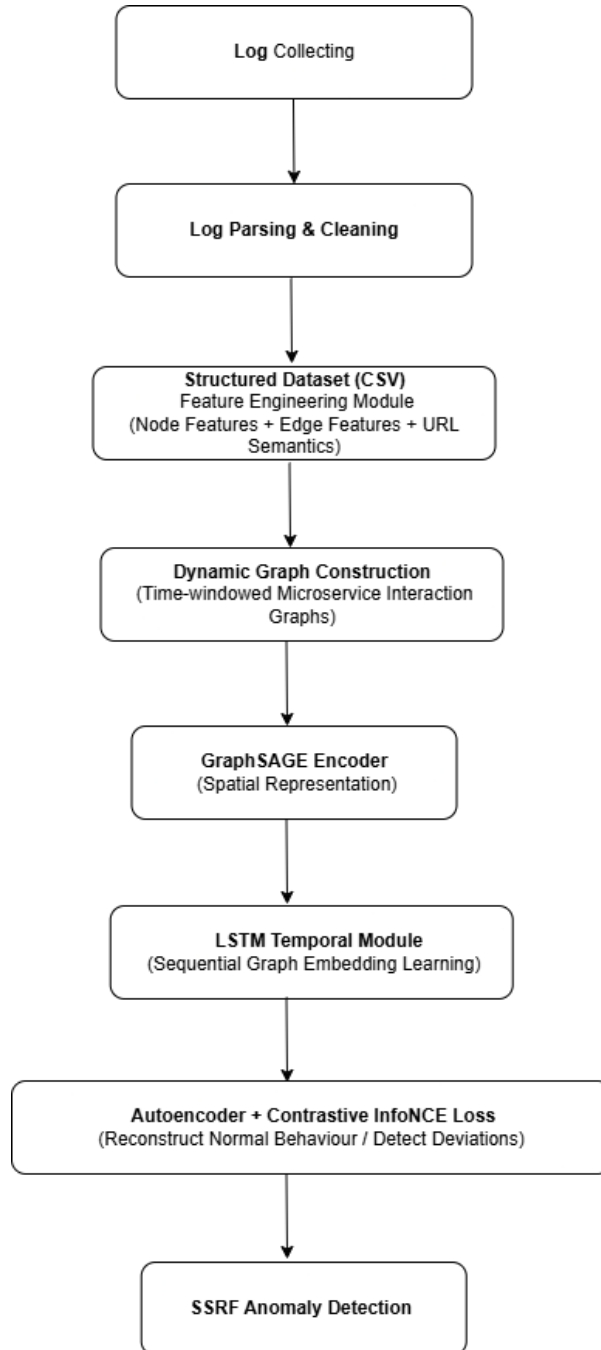


Figure 3.1: Conceptual System Architecture for Unsupervised SSRF Detection

CHAPTER 4

IMPLEMENTATION

The implementation of this project comprises two major components. The first component involves deploying a realistic microservice environment and generating authentic service-to-service communication logs using Kubernetes and Minikube. The second component involves designing and executing a complete data processing pipeline that transforms raw logs into structured datasets suitable for spatio-temporal analysis in future SSRF detection systems. This chapter presents an exhaustive description of each step in the implementation process, elaborating on the technical decisions, the workflow, and the rationale guiding the transformations from raw logs to engineered feature datasets.

The implementation described in this chapter reflects the work actually completed: deployment of the Google Online Boutique microservice architecture, generation of traffic using a load generator, extraction of logs from multiple microservices, parsing the consolidated log file, constructing a structured dataset, and engineering node-level and edge-level features. While the design of the complete Graph Neural Network and LSTM-based SSRF detection architecture is included conceptually in the methodology chapter, only dataset construction and feature engineering were implemented in this phase.

4.1 Microservice Environment Deployment

The implementation began with the deployment of the Google Cloud Online Boutique microservice application. This application is widely used as a benchmark system for microservice research due to its realistic multi-service structure, diverse internal communication patterns, and well-documented architecture. The application consists of distinct microservices such as the frontend service, product catalog service, recommendation service, checkout service, payment service, shipping service, currency service, and several others. Each microservice exposes

one or more HTTP endpoints and communicates with other services through internal requests.

To deploy the environment in a controlled and reproducible manner, Minikube was chosen as the Kubernetes distribution. Minikube allows containerized microservices to run locally while still providing the core capabilities of Kubernetes, including pod scheduling, service resolution, internal networking, and logging functionality. The deployment process began with initializing Minikube using the Docker driver and ensuring that the Kubernetes cluster was fully online before applying the Online Boutique manifests.

The Kubernetes manifest files for the Online Boutique architecture were then applied using the `kubectl apply` command. These manifests created deployments, services, and pods corresponding to each microservice. After deployment, the system was monitored using `kubectl get pods` to verify that all microservices had entered the “Running” state. This ensured that service-to-service communication could occur without interference and that the system was functioning as expected.

Once the microservice architecture was operational, Minikube’s service routing was used to expose the frontend web interface locally. This allowed confirmation that user interactions triggered the appropriate backend service calls and that the microservices were communicating internally as designed. This stage was crucial because the quality and realism of the logs depended entirely on the correctness of the deployment.

4.2 Traffic Generation Using Loadgenerator

After confirming the stability of the microservice architecture, traffic generation was initiated through the deployment of the load generator provided with the Online Boutique system. The load generator is responsible for simulating realistic user behaviour, such as browsing products, viewing item details, adding products to the shopping cart, and proceeding through the checkout workflow.

The load generator operates continuously and repeatedly triggers HTTP requests to the frontend service, which then forwards requests internally to other microservices. These internal requests form the basis of the log data used in this project. If insufficient traffic is generated, microservices will produce too few log entries, resulting in a dataset that does not accurately reflect real-world service patterns.

The load generator deployment was applied using its predefined Kubernetes manifest. Once running, the load generator immediately started producing traffic. Logs from the microservices

confirmed that internal requests were being issued frequently and consistently. This continuous load not only produced a rich log file but also ensured that services interacted in various patterns, covering a wide variety of request types and URL endpoints.

The load generator continued to run for an extended period to ensure adequate collection of logs. Longer traffic generation allowed the system to capture stable behaviour, variations caused by load fluctuations, and repeated interactions among microservices. This resulted in a comprehensive log file that reflected natural microservice communication.

4.3 Log Extraction and Consolidation

After sufficient traffic was generated, logs from all microservices were extracted. Kubernetes provides convenient access to the standard output of each container through the `kubectl logs` command. Using deployment labels and pod names, logs were collected individually from each microservice.

These logs include detailed request traces containing fields such as timestamps, HTTP methods, URL paths, response codes, latency values, and payload sizes. Each service logs entries about the requests it receives and, depending on implementation, the outbound requests it forwards to other services.

To simplify the data processing pipeline, the extracted logs from all microservices were consolidated into a single log file. Consolidation ensures easier parsing and enables unified dataset construction without the need to merge or align multiple log sources manually. This consolidated log file served as the raw input for the Python-based preprocessing pipeline.

4.4 Python-Based Log Parsing Pipeline

The consolidated log file was parsed using a custom Python script. This script was designed to extract structured fields from each log entry using regular expressions. Because the logs were semi-structured, regular expressions allowed precise extraction of fields such as the timestamp, source service name, destination service name, HTTP method, URL path, status code, latency, and payload size.

Parsing involved iterating over each line in the raw log file and applying the matching pattern. Only lines that matched the expected format were processed, while malformed or incomplete lines were skipped to maintain dataset quality. Each successfully parsed log entry

was stored as a dictionary containing all extracted fields.

After parsing all lines, these dictionaries were converted into a Pandas DataFrame. This DataFrame represented the complete request-level dataset and served as the foundation for all subsequent analysis. The structure of the DataFrame ensured compatibility with grouping, filtering, and aggregation operations required for feature extraction.

During this stage, the entries were also validated for consistency. Any entries with missing fields or invalid numerical values were removed. Timestamps were converted into sortable formats to preserve chronological ordering, which is essential for any future temporal analysis.

The parsed dataset was then exported as `extracted_dataset.csv`. This file contains one row per request and provides a high-fidelity representation of microservice interactions.

4.5 Dataset Structuring and Cleaning

The request-level DataFrame was examined for inconsistencies such as duplicate entries, irregular timestamp formats, or unexpected service name variations. Service names were standardized to ensure uniform grouping across features. Latency values were checked to ensure they conformed to expected numerical ranges, and payload sizes were sanitized to remove any noise.

This cleaning step was essential because inaccurate data would distort the statistical features derived later. For instance, erroneous high latency values could inflate averages, while incorrect service names could create artificial nodes in the graph representation. As a result, careful attention was given to data quality during this stage.

Once cleaned, the structured dataset represented a complete and accurate record of service-to-service communication.

4.6 Node-Level Feature Engineering

Node-level features were extracted by grouping the dataset based on the name of the microservice involved in each request, either as a source or as a destination. For each microservice, all associated log entries were aggregated to form a behavioural summary.

Key metrics such as total number of interactions, incoming request count, outgoing request count, average latency, maximum and minimum latency, error rate, average payload size, and URL diversity were computed. These metrics offer a high-level characterization of each mi-

croservice.

Node-level features reflect how each service behaves within the overall system. For example, the frontend service typically initiates a large number of requests because user interaction flows begin there. Service roles can also be inferred from behaviour: `paymentservice` often receives requests from `checkoutservice`, while `recommendationservice` handles user-specific suggestions.

These node-level features were stored in a new DataFrame and exported as `node_features.csv`. This file provides the node attribute matrix required for graph-based learning.

4.7 Edge-Level Feature Engineering

Edge-level features were extracted by grouping the dataset based on source–destination service pairs. These features represent the characteristics of communication between specific microservices.

For each unique service pair, metrics such as call frequency, average latency, maximum latency, minimum latency, error rate, average payload size, and the number of unique URLs accessed were computed. Additionally, URL semantic indicators were aggregated across each service pair.

Edge-level features provide deep insight into system communication. For example, `checkoutservice` consistently communicates with `paymentservice` and `shippingservice`. `Frontendservice` frequently communicates with `productcatalogservice`. Deviations from these patterns in future logs would indicate irregular behaviour, making edge features essential for SSRF detection models.

The resulting DataFrame was exported as `edge_features.csv`, representing the edge attribute matrix required for graph-based modelling.

4.8 Saving Feature Files and Dataset Outputs

The project produced three major output files:

1. `extracted_dataset.csv`: the full request-level dataset.
2. `node_features.csv`: the microservice-level behavioural summaries.
3. `edge_features.csv`: the service-pair interaction summaries.

These files are essential artifacts that support future work in constructing graph structures, designing GNN architectures, and performing temporal modelling.

4.9 Conceptual Design of SSRF Detection Architecture

Although this project focused on dataset creation, the implementation work was guided by the conceptual design of a larger SSRF detection system. The dataset built here is intended to serve as the input for a future unsupervised anomaly detection model combining GraphSAGE for spatial feature learning and LSTM networks for temporal modelling.

The conceptual architecture includes stages such as constructing dynamic graph snapshots from logs, generating node and edge embeddings, applying LSTM layers to capture temporal dependencies, and using autoencoder reconstruction error to flag anomalies. While these components were not implemented, the design influenced how the dataset was structured and which features were extracted.

The extracted features ensure compatibility with future GNN frameworks, enabling smooth transition into spatio-temporal graph learning tasks.

4.10 Summary

This chapter provided a comprehensive and detailed account of the implementation performed in this project. It covered the deployment of the microservice environment, generation of internal communication logs, extraction and consolidation of raw log data, parsing and structuring of the log entries, engineering of node-level and edge-level features, and preparation of datasets suitable for future machine learning tasks. The outputs of this implementation form the essential foundation required for advanced SSRF detection using graph neural networks and temporal modelling techniques.

CHAPTER 5

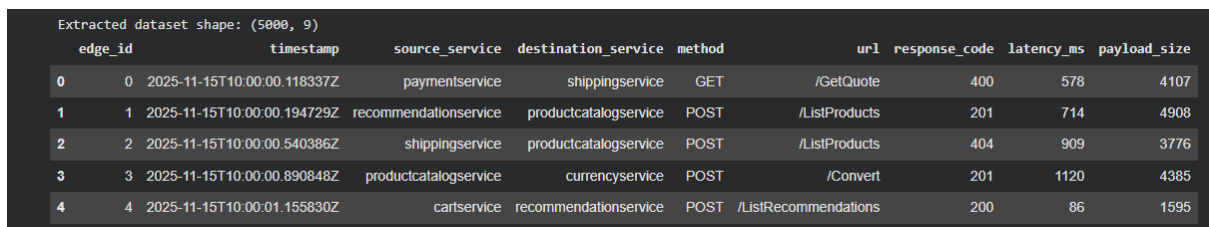
RESULTS AND ANALYSIS

The implementation phase of this work produced a structured and feature-rich dataset derived from the internal communication logs of a microservice-based application. The results presented in this chapter focus on the outputs generated: the consolidated log file, the structured request-level dataset, the node-level behavioural summary, and the edge-level communication feature matrix. These datasets form the foundational components required for future graph-based SSRF anomaly detection research.

5.1 Request-Level Dataset Output

The first major output produced was the structured request-level dataset generated from the raw Kubernetes logs. Each row in the dataset corresponds to a single HTTP request that occurred between two microservices. The dataset includes the timestamp of the request, the source and destination service, the HTTP method, URL path, response status code, request latency, and payload size. This structured format allows for efficient analysis and feature extraction.

A representative portion of the structured dataset is shown in Figure 5.1, illustrating how raw log messages were transformed into clean, machine-readable tabular entries.



Extracted dataset shape: (5000, 9)									
	edge_id	timestamp	source_service	destination_service	method	url	response_code	latency_ms	payload_size
0	0	2025-11-15T10:00:00.118337Z	paymentservice	shippingservice	GET	/GetQuote	400	578	4107
1	1	2025-11-15T10:00:00.194729Z	recommendationservice	productcatalogservice	POST	/ListProducts	201	714	4908
2	2	2025-11-15T10:00:00.540386Z	shippingservice	productcatalogservice	POST	/ListProducts	404	909	3776
3	3	2025-11-15T10:00:00.890848Z	productcatalogservice	currencyservice	POST	/Convert	201	1120	4385
4	4	2025-11-15T10:00:01.155830Z	cartservice	recommendationservice	POST	/ListRecommendations	200	86	1595

Figure 5.1: Extracted Request-Level Dataset After Parsing the Kubernetes Log

This dataset captures stable communication patterns among microservices, enabling the identification of deviations that may indicate SSRF-like anomalies. The dataset is also chrono-

logically sorted, ensuring compatibility with future temporal modeling techniques.

5.2 Kubernetes Log Output

Before parsing, the microservices produced raw textual logs representing internal HTTP communications. These logs contain the original unstructured request traces with timestamps, source and destination identifiers, latency messages, and response codes.

A segment of the collected raw Kubernetes log file is shown in Figure 5.2. This figure demonstrates the complexity and unstructured nature of the log entries before processing.

```
2025-11-15T10:00:00.194729Z adservice-mum0o adservice-container [recommendationservice] Sent request to productcatalogservice: POST /ListProducts; response=201; latency=714ms; size=49088
2025-11-15T10:00:00.540386Z adservice-mum0o adservice-container [shippingservice] Sent request to productcatalogservice: POST /ListProducts; response=404; latency=909ms; size=3776B
2025-11-15T10:00:00.540386Z adservice-mum0o adservice-container WARN [shippingservice] High latency detected: 909ms calling productcatalogservice
2025-11-15T10:00:00.890848Z adservice-mum0o adservice-container [productcatalogservice] Sent request to currencyservice: POST /Convert; response=201; latency=1120ms; size=4385B
2025-11-15T10:00:00.890848Z adservice-mum0o adservice-container WARN [productcatalogservice] High latency detected: 1120ms calling currencyservice
2025-11-15T10:00:01.155830Z adservice-mum0o adservice-container [cartservice] Sent request to recommendationservice: POST /ListRecommendations; response=200; latency=86ms; size=1595B
2025-11-15T10:00:01.393580Z adservice-mum0o adservice-container [recommendationservice] Sent request to paymentservice: POST /Charge; response=404; latency=1204ms; size=4915B
2025-11-15T10:00:01.393580Z adservice-mum0o adservice-container WARN [recommendationservice] High latency detected: 1204ms calling paymentservice
2025-11-15T10:00:01.934311Z adservice-mum0o adservice-container [checkoutservice] Sent request to paymentservice: POST /Charge; response=500; latency=957ms; size=3707B
2025-11-15T10:00:01.934311Z adservice-mum0o adservice-container WARN [checkoutservice] High latency detected: 957ms calling paymentservice
2025-11-15T10:00:01.934311Z adservice-mum0o adservice-container ERROR [checkoutservice] Upstream error from paymentservice: HTTP 500
2025-11-15T10:00:02.294126Z adservice-mum0o adservice-container [emailservice] Sent request to cartservice: POST /GetCart; response=404; latency=541ms; size=1783B
2025-11-15T10:00:02.327779Z adservice-mum0o adservice-container [recommendationservice] Sent request to paymentservice: GET /Charge; response=200; latency=1253ms; size=2805B
2025-11-15T10:00:02.327779Z adservice-mum0o adservice-container WARN [recommendationservice] High latency detected: 1253ms calling paymentservice
2025-11-15T10:00:02.867825Z adservice-mum0o adservice-container [shippingservice] Sent request to checkoutservice: POST /PlaceOrder; response=500; latency=1273ms; size=4187B
2025-11-15T10:00:02.867825Z adservice-mum0o adservice-container WARN [shippingservice] High latency detected: 1273ms calling checkoutservice
2025-11-15T10:00:02.867825Z adservice-mum0o adservice-container ERROR [shippingservice] Upstream error from checkoutservice: HTTP 500
2025-11-15T10:00:02.881218Z adservice-mum0o adservice-container [cartservice] Sent request to currencyservice: POST /Convert; response=400; latency=107ms; size=3722B
2025-11-15T10:00:03.282295Z adservice-mum0o adservice-container [checkoutservice] Sent request to productcatalogservice: POST /ListProducts; response=201; latency=333ms; size=3041B
2025-11-15T10:00:03.467960Z adservice-mum0o adservice-container [emailservice] Sent request to shippingservice: GET /GetQuote; response=404; latency=826ms; size=3965B
2025-11-15T10:00:03.467960Z adservice-mum0o adservice-container WARN [emailservice] High latency detected: 826ms calling shippingservice
2025-11-15T10:00:03.959702Z adservice-mum0o adservice-container [cartservice] Sent request to frontend: GET /Cart; response=403; latency=1403ms; size=2532B
2025-11-15T10:00:03.959702Z adservice-mum0o adservice-container WARN [cartservice] High latency detected: 1403ms calling frontend
2025-11-15T10:00:04.407008Z adservice-mum0o adservice-container [recommendationservice] Sent request to checkoutservice: POST /PlaceOrder; response=200; latency=465ms; size=436B
2025-11-15T10:00:04.658643Z adservice-mum0o adservice-container [currencyservice] Sent request to checkoutservice: GET /PlaceOrder; response=200; latency=486ms; size=
```

Figure 5.2: Raw Kubernetes Log Capturing Microservice-to-Microservice Communication

This log file served as the primary input for parsing and dataset creation.

5.3 Node-Level Feature Output

Node features summarize the behavioural characteristics of each microservice by aggregating all incoming and outgoing requests associated with that service. These features include the total number of calls, incoming call count, outgoing call count, average latency, maximum and minimum latency, error rate, number of unique URLs, average payload size, and slow-call ratio.

A sample of the generated node feature table is shown in Figure 5.3. Each row corresponds to a microservice in the application.

These node features provide an overview of service behaviour and will serve as node attributes in future graph-based anomaly detection models.

	service	total_calls	outgoing_calls	incoming_calls	avg_latency	max_latency	min_latency	error_rate	unique_urls	avg_payload	slow_d
0	paymentservice	993	476	517	768.169184	1498	22	0.166163	15	2584.278953	
1	shippingservice	1001	529	472	760.247752	1498	22	0.163836	15	2608.678322	
2	recommendationservice	977	512	465	763.056295	1500	21	0.172979	15	2567.234391	
3	productcatalogservice	989	460	529	774.658241	1497	21	0.179980	15	2632.892821	
4	currencyservice	976	496	480	763.669057	1494	20	0.158811	15	2535.244877	

Figure 5.3: Node-Level Behavioural Features Extracted from the Structured Dataset

5.4 Edge-Level Feature Output

Edge features capture the behavioural characteristics of communication between pairs of microservices. For each source–destination pair, metrics such as call frequency, average latency, maximum and minimum latency, error rate, unique URL count, average payload size, and URL semantic indicators were extracted.

Figure 5.4 presents a sample of the edge feature matrix generated during feature engineering.

	source_service	destination_service	call_count	avg_latency	max_latency	min_latency	error_rate	avg_payload	unique_urls	level_2	url
0	adservice	cartservice	57	757.754386	1494	27	0.105263	2615.035088	2	url_length	512
1	adservice	cartservice	57	757.754386	1494	27	0.105263	2615.035088	2	slash_count	57
2	adservice	cartservice	57	757.754386	1494	27	0.105263	2615.035088	2	has_ip	0
3	adservice	cartservice	57	757.754386	1494	27	0.105263	2615.035088	2	has_internal	0
4	adservice	cartservice	57	757.754386	1494	27	0.105263	2615.035088	2	suspicious	0

Figure 5.4: Edge-Level Communication Features for Microservice Interactions

These features represent the relational structure of the microservice system and will form the basis for constructing graph edges and edge attribute matrices in later stages of SSRF anomaly detection research.

5.5 Summary of Extracted Results

The outputs generated in this implementation stage include:

- The raw Kubernetes log file representing unprocessed microservice communication.

- A fully structured request-level dataset suitable for graph construction and temporal sequence modelling.

- Node-level feature vectors capturing the behavioural profile of each microservice.

- Edge-level feature vectors describing communication properties between pairs of services.

Together, these datasets provide a comprehensive and high-quality foundation for building graph-based SSRF detection mechanisms in future work. They preserve both the structural and semantic characteristics required for GNN and LSTM-based anomaly detection approaches.

CHAPTER 6

CONCLUSION AND FUTURE WORK

The increasing reliance on microservice-based architectures in cloud-native environments has amplified both the operational benefits and the security challenges associated with distributed computing. As organizations embrace the flexibility and scalability offered by microservices, they simultaneously expose internal service endpoints, inter-service communication channels, and metadata resources that may not have been intended for public access. Among the many security threats targeting such environments, Server-Side Request Forgery (SSRF) stands out as one of the most critical and difficult-to-detect vulnerabilities. Unlike conventional attacks that originate externally and target public endpoints, SSRF manipulates backend services into interacting with unauthorized internal destinations, making detection an inherently complex task.

The overarching objective of this project was to construct a foundational dataset that can support future research in graph-based and temporal SSRF detection. While the eventual goal is to enable unsupervised anomaly detection using Graph Neural Networks and LSTM models, this work focuses on the essential building blocks required to achieve that objective. The implementation successfully deployed a real microservice environment, generated realistic traffic, collected detailed logs, parsed the unstructured data, and engineered comprehensive node- and edge-level features. These components form the structural, semantic, and behavioural representations necessary for building advanced anomaly detection frameworks.

The first major accomplishment of this work lies in the systematic collection of microservice logs reflecting real service-to-service interactions. These logs form the raw behavioural footprint of the system and are vital for any SSRF detection methodology. The subsequent parsing process transformed noisy, unstructured log entries into a precise and well-organized dataset. By extracting timestamps, HTTP methods, URL paths, service identifiers, latency values, payload sizes, and response codes, the project ensured that critical details necessary for

modelling communication patterns were preserved.

Feature engineering represents another significant contribution of the work. Node-level features summarize the behavioural tendencies of microservices, revealing how frequently they communicate, how quickly they respond, and how often errors occur. These features encapsulate service-centric properties and help distinguish normal behaviour from anomalies. Edge-level features, on the other hand, capture the dynamics of inter-service communication. They provide a deeper understanding of which services interact, how frequently they do so, and under what performance characteristics. Such interaction-focused features are essential for detecting SSRF-like deviations where a service begins communicating with unexpected or unauthorized endpoints.

The inclusion of semantic URL features adds an additional layer of insight. Since SSRF attacks often rely on manipulating URLs to access internal endpoints or sensitive cloud metadata, analysing URL characteristics such as length, host patterns, depth, and suspicious keywords becomes invaluable. These features enrich the dataset by incorporating behavioural indicators that are difficult to infer from latency or payload data alone.

Overall, the dataset produced through this project is well-structured, highly detailed, and aligned with the requirements identified in recent literature. Researchers have repeatedly emphasized the need for rich, multi-dimensional data when performing anomaly detection in microservice environments. The dataset constructed here satisfies these requirements, making it a viable foundation for future work involving deep learning and graph-based analysis.

The limitations of this work primarily stem from its scope. While the dataset provides all of the necessary inputs for graph and temporal models, the implementation does not extend to the machine learning or anomaly detection stage. This decision was deliberate, as the primary focus was on building a reusable, extensible, and high-quality dataset. Furthermore, SSRF attacks are rare in real-world datasets, and the logs generated by the Online Boutique environment represent entirely normal operation. Thus, no malicious behaviour is naturally present in the dataset. Future work will require either synthetic injection of SSRF-like patterns or access to real-world SSRF datasets to train and evaluate detection models effectively.

The future direction for this project is both clear and promising. The first step involves constructing a dynamic graph sequence from the dataset, grouping interactions into time windows to create evolving graph snapshots. These snapshots can then be used as input for Graph Neural Networks such as GraphSAGE, which can learn structural embeddings from node and

edge features. Once spatial embeddings are obtained, an LSTM or GRU network can be applied to capture temporal dynamics and learn how microservice behaviour evolves over time. The combined GNN–LSTM framework is expected to detect deviations in both structure and sequence, thereby identifying SSRF-like anomalies that standalone models may miss.

Another direction for future research involves incorporating autoencoder-based reconstruction models for unsupervised anomaly detection. Since labelled SSRF datasets are scarce, unsupervised approaches are essential. Reconstruction error provides a powerful signal for identifying anomalies because the model learns to reproduce normal behaviour but fails when confronted with unfamiliar patterns. This makes it ideal for SSRF, where payloads and request targets may deviate drastically during an attack.

Future enhancements may also involve integrating service mesh telemetry, container metrics, and Kubernetes events into the dataset. These additional dimensions provide richer contextual information and allow the development of multimodal anomaly detection systems. Moreover, incorporating metadata endpoints, privilege escalation logs, and internal cloud service interactions could substantially improve SSRF detection accuracy.

Finally, deploying the system in a real-time monitoring environment represents a long-term research goal. Real-time SSRF detection requires efficient stream processing, incremental graph updates, and near-real-time inference from the GNN–LSTM model. Achieving this would enable organizations to detect SSRF attacks as they unfold, thereby preventing severe damage caused by unauthorized internal access.

In conclusion, this project lays the groundwork for graph-based SSRF detection by successfully constructing an extensive, feature-rich dataset derived from genuine microservice interactions. While the anomaly detection model remains a task for future research, the dataset created here aligns with the structural and semantic requirements identified in state-of-the-art literature. This positions the work as a strong foundation for continued exploration in the field of microservice security, graph analytics, and unsupervised anomaly detection.

REFERENCES

- [1] Khadejah Al-talak and Dr. Onytra Abbass. Detecting server-side request forgery (ssrf) attack by using deep learning techniques. *International Journal of Computer Applications*, 2021.
- [2] Shuai Fang, Jingjian Chao, Zenghui Xiang, Xuan Chen, Mei Yan, and Yinan Dou. Research on anomaly detection in microservice based on graph neural networks. *Journal of Systems and Software*, 2024.
- [3] Jalal Ghadermazi, Soumyadeep Hore, Ankit Shah, and Nathaniel D. Bastian. Gtae-ids: Graph transformer-based autoencoder framework for real-time network intrusion detection. *IEEE Transactions on Information Forensics and Security*, 2025.
- [4] Ming Jin, Huan Yee Koh, Qingsong Wen, Daniele Zambon, Cesare Alippi, Geoffrey I. Webb, Irwin King, and Shirui Pan. A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection. *Information Sciences*, 2022.